

TCS - 100 Technical Interview Questions

Tata Consultancy Services

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. What is operator overloading? Give an example in C++.

Answer:

Operator overloading lets you redefine what an operator like '+' does for a user-defined type. For example, in C++ you could overload '+' for a Complex number class so that `c1 + c2` adds their real and imaginary parts, letting Complex objects be used with familiar arithmetic syntax.

Q2. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q3. Explain the SOLID principles of object-oriented design.

Answer:

SOLID stands for: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable for their base types without breaking correctness), Interface Segregation (prefer many small specific interfaces over one large one), and Dependency Inversion (depend on abstractions, not concrete implementations).

Q4. What is a functional interface in Java, and how does it relate to lambda expressions?

Answer:

A functional interface is an interface with exactly one abstract method (it may have default/static methods too), e.g. `Runnable` or `Comparator`. Java's lambda expressions are essentially shorthand implementations of a functional interface's single abstract method, letting you write `(a, b) -> a.compareTo(b)` instead of an anonymous inner class.

Q5. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q6. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

Q7. Explain the difference between 'is-a' and 'has-a' relationships with examples.

Answer:

An 'is-a' relationship implies inheritance - a Dog is-a Animal. A 'has-a' relationship implies composition - a Car has-a Engine. Choosing 'has-a' over 'is-a' when there isn't a true specialization relationship avoids incorrectly forcing an inheritance hierarchy where composition would be more appropriate.

Q8. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q9. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q10. What is object cloning, and what is the difference between a shallow copy and a deep copy?

Answer:

Object cloning creates a copy of an existing object. A shallow copy duplicates the object's top-level fields but nested objects/references are shared with the original (mutating a nested object affects both copies). A deep copy recursively duplicates nested objects too, so the copy is fully independent of the original.

Q11. What happens when a class implements an interface but does not implement all its methods?

Answer:

If a class implements an interface but doesn't implement all its abstract methods, the class itself must be declared abstract; otherwise, it is a compile-time error since an interface is a contract that guarantees every method has an implementation somewhere in the concrete class hierarchy.

Q12. Explain the concept of a virtual function in C++ and how it enables runtime polymorphism.

Answer:

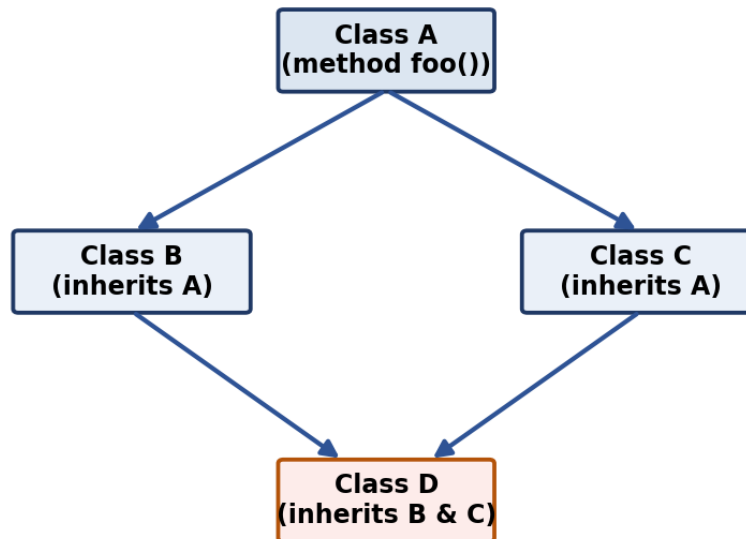
A virtual function in C++ is declared with the 'virtual' keyword in a base class, telling the compiler to resolve the call at runtime via a vtable based on the actual object type rather than the reference/pointer type. This is what allows a Shape* pointing to a Circle to correctly call Circle's draw() instead of Shape's - the mechanism behind runtime polymorphism in C++.

Q13. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.

Diamond Problem: which foo() does D inherit - via B or via C?



Q14. Explain the concept of late binding vs early binding with an example.

Answer:

Early (static) binding resolves which method to call at compile time, based on the reference type - used for overloaded and static methods. Late (dynamic) binding resolves the method call at runtime, based on the actual object type - used for overridden instance methods, which is the basis of runtime polymorphism.

DBMS & SQL

Q15. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q16. What is indexing, and what are the trade-offs of adding too many indexes to a table?

Answer:

An index is a data structure (commonly a B-Tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT queries. The trade-off is that every INSERT/UPDATE/DELETE must also update all indexes on that table, slowing down writes and consuming extra storage - so indexes should be added selectively on columns that are frequently filtered or joined on.

Q17. Explain database isolation levels and the anomalies (dirty read, non-repeatable read, phantom read) each one prevents.

Answer:

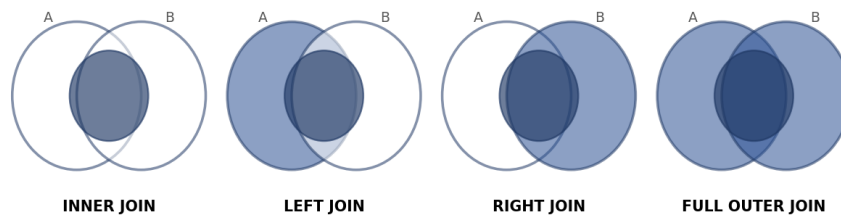
Isolation levels control how much one transaction can see of another's uncommitted work: Read Uncommitted allows dirty reads (seeing uncommitted changes); Read Committed prevents dirty reads but allows non-repeatable reads (a value changes between two reads in the same transaction); Repeatable Read prevents that but allows phantom reads (new rows appearing in a repeated range query); Serializable prevents all three by fully isolating transactions, at the cost of concurrency.

Q18. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN with examples.

Answer:

INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table plus matched rows from the right (NULLs where there's no match). RIGHT JOIN is the mirror - all rows from the right table plus matches from the left. FULL OUTER JOIN returns all rows from both tables, with NULLs wherever a match is missing on either side.

SQL JOIN Types (shaded = rows returned)



Q19. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q20. Explain the difference between a primary key, a unique key, and a candidate key.

Answer:

A candidate key is any column (or set of columns) that could uniquely identify a row - a table can have several. A primary key is the one candidate key chosen to be the table's main unique identifier (cannot be NULL). A unique key is a constraint enforcing uniqueness on a column that is not the primary key, and unlike a primary key, it typically allows one NULL value.

Q21. What is denormalization, and when would you intentionally denormalize a schema?

Answer:

Denormalization intentionally introduces redundancy (e.g. storing a customer's name directly on an Orders row instead of always joining to a Customers table) to reduce the number of joins needed and speed up read-heavy queries, at the cost of extra storage and more complex writes to keep the redundant copies in sync. It's used in reporting/analytics systems where read performance matters more than write simplicity.

Q22. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age $\geq$ 18), independent of any other table.

Q23. What is a stored procedure, and what are its advantages over writing ad-hoc queries?

Answer:

A stored procedure is a precompiled, named collection of SQL statements stored in the database and invoked as a single call. Advantages include reduced network round-trips (one call instead of many statements), better security (grant execute rights without exposing underlying tables directly), reusability across applications, and a cached execution plan that can improve performance over repeatedly sent ad-hoc queries.

Q24. Explain the difference between a correlated subquery and a regular subquery.

Answer:

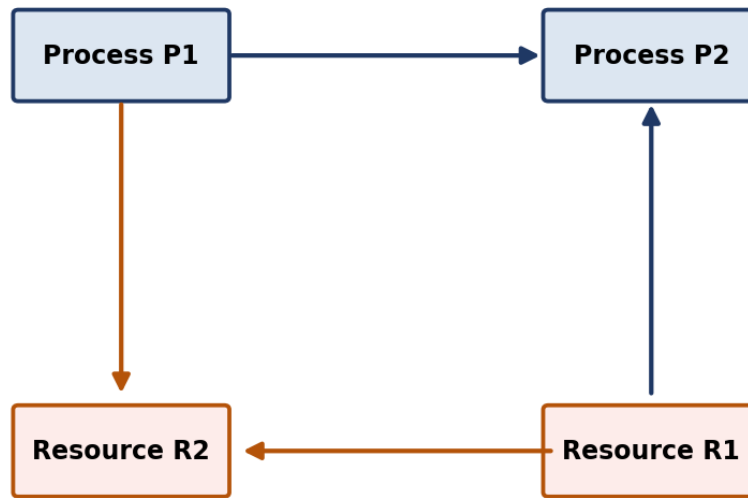
A regular (uncorrelated) subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query and is conceptually re-evaluated for each row processed by the outer query, e.g. finding employees who earn more than the average salary of their own department.

Q25. What is a deadlock in a database, and how can it be prevented or resolved?

Answer:

A database deadlock happens when two or more transactions each hold a lock the other needs and wait on each other indefinitely (e.g. Transaction A locks Row 1 and waits for Row 2, while Transaction B locks Row 2 and waits for Row 1). It can be prevented by always acquiring locks in a consistent order across transactions, kept short by minimizing transaction scope, or resolved by the database's deadlock detector automatically rolling back one transaction (the 'victim') so the other can proceed.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q26. Write a SQL query using GROUP BY and HAVING to find departments with more than 5 employees.

Answer:

SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 5. GROUP BY collapses rows into per-department groups, and HAVING filters those groups (after aggregation) to keep only departments with more than five employees.

Q27. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q28. What is a view in SQL, and how is it different from a materialized view?

Answer:

A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q29. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Q30. What is sharding, and how does it help scale a database horizontally?

Answer:

Sharding splits a large table's rows horizontally across multiple database servers (shards), typically based on a shard key (e.g. customer_id range or hash), so no single machine needs to store or query the entire dataset. It improves write and storage scalability but introduces challenges like cross-shard joins/transactions and rebalancing when adding new shards.

Q31. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

```
SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department).
```

The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q32. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Operating Systems

Q33. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q34. What is a semaphore? Differentiate between binary and counting semaphores.

Answer:

A semaphore is an integer variable used for process synchronization, with atomic increment (signal/V) and decrement (wait/P) operations. A binary semaphore takes only values 0 or 1 and behaves like a mutex, ensuring exclusive access. A counting semaphore can take any non-negative integer, used to manage a pool of multiple identical resources (e.g. a fixed number of database connections).

Q35. What is the difference between a hard link and a symbolic link in a file system?

Answer:

A hard link is essentially another directory entry pointing directly to the same inode (and underlying data) as the original file - deleting the original name still leaves the data accessible via the hard link, and it cannot span file systems. A symbolic (soft) link is a small separate file that stores the path to the target file - it can point across file systems or to directories, but becomes a 'dangling link' if the target is deleted or moved.

Q36. Explain the difference between preemptive and non-preemptive scheduling with examples of each.

Answer:

Preemptive scheduling allows the OS to forcibly suspend a running process to give the CPU to another (e.g. Round Robin, Priority Scheduling with preemption) - responsive but has context-switch overhead. Non-preemptive scheduling lets a running process keep the CPU until it voluntarily yields or finishes (e.g. FCFS, non-preemptive SJF) - simpler but a long process can block shorter ones ('convoy effect').

Q37. What is the difference between internal and external fragmentation?

Answer:

Internal fragmentation is wasted space inside an allocated fixed-size block that's larger than what the process actually needs (common in paging). External fragmentation is wasted space between allocated blocks - free memory exists in total but is scattered in pieces too small individually to satisfy a new request (common in segmentation/variable-size allocation).

Q38. What is a critical section, and what conditions must a solution to the critical-section problem satisfy?

Answer:

A critical section is a segment of code that accesses shared resources and must not be executed by more than one process/thread concurrently to avoid race conditions. A valid solution must guarantee: Mutual Exclusion (only one process in the critical section at a time), Progress (a process not in the critical section can't block others from entering), and Bounded Waiting (a limit on how many times other processes can enter before a waiting process gets its turn).

Q39. What is a context switch, and why does it have a performance cost?

Answer:

A context switch is the process of saving the CPU state (registers, program counter) of a currently running process/thread and loading the saved state of another, so execution can resume later exactly where it left off. It has a performance cost because the CPU does no useful work during the switch, and it can also cause cache/TLB misses afterward as the new process's data isn't cached yet.

Q40. Explain how a page fault is handled by the operating system.

Answer:

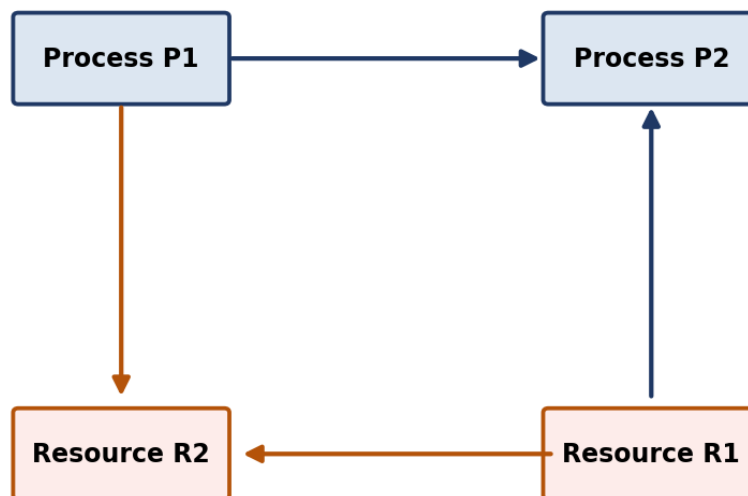
When a process accesses a page not currently in physical memory, the CPU raises a page fault trap. The OS then: locates the required page on disk, finds a free frame (or selects a victim page to evict via a page replacement algorithm like LRU, writing it back to disk if modified), loads the required page into that frame, updates the page table, and resumes the instruction that caused the fault.

Q41. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q42. Explain the difference between a monolithic kernel and a microkernel.

Answer:

A monolithic kernel runs all core OS services (file system, device drivers, memory management, scheduling) in a single address space in kernel mode, which is fast (no IPC overhead between components) but a bug in any component can crash the whole system. A microkernel keeps only the bare minimum (IPC, basic scheduling, memory management) in kernel space and moves other services to user-space processes, improving fault isolation and modularity at the cost of more inter-process communication overhead.

Q43. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Q44. What is virtual memory, and why is it useful even when physical RAM is limited?

Answer:

Virtual memory gives each process the illusion of a large, contiguous address space by mapping logical addresses to physical RAM or disk (swap space) via page tables, transparently to the program. It's useful even with limited RAM because it allows programs larger than physical memory to run, isolates each process's memory from others for safety, and lets the OS overcommit memory across many processes that don't all need their full memory at once.

Q45. Explain how the Least Recently Used (LRU) page replacement algorithm works, and how you might implement it efficiently.

Answer:

LRU (Least Recently Used) evicts the page that hasn't been used for the longest time when a new page must be loaded and memory is full, based on the assumption that recently used pages are likely to be used again soon. An efficient implementation uses a combination of a hashmap (for $O(1)$ lookup of a page's location) and a doubly linked list ordered by recency, so both finding a page and moving it to the 'most recently used' end take $O(1)$ time.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q46. Compare FCFS, SJF, Round Robin, and Priority scheduling algorithms - what are the trade-offs of each?

Answer:

FCFS (First-Come-First-Served) is simple but can cause the 'convoy effect' where a long job delays all others. SJF (Shortest Job First) minimizes average waiting time but requires knowing job lengths in advance and can starve long jobs. Round Robin gives each process a fixed time quantum, ensuring fairness and responsiveness for interactive systems, but too small a quantum increases context-switch overhead. Priority Scheduling runs the highest-priority job first but can starve low-priority jobs unless aging is used.

Computer Networks

Q47. What is NAT, and why is it necessary given the limited IPv4 address space?

Answer:

NAT (Network Address Translation) allows multiple devices on a private network to share a single public IP address when communicating with the internet, translating private IPs to the public IP (and back for return traffic) at the router/gateway. It's necessary because IPv4's roughly 4.3 billion addresses are far fewer than the number of internet-connected devices worldwide.

Q48. Explain what a firewall does and the difference between a stateful and stateless firewall.

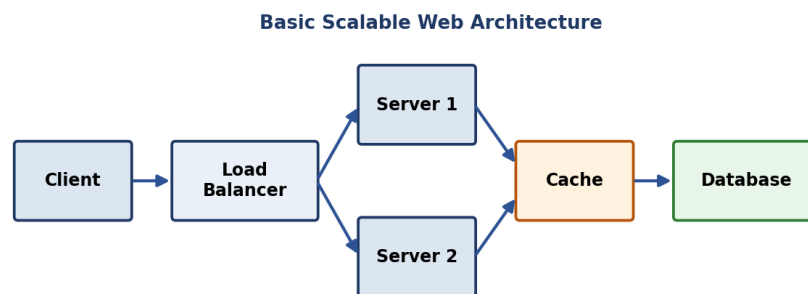
Answer:

A firewall monitors and controls incoming/outgoing network traffic based on defined security rules. A stateless firewall evaluates each packet independently against static rules (e.g. block port 23), without knowledge of ongoing connections. A stateful firewall tracks the state of active connections and can make smarter decisions, like automatically allowing return traffic for a connection that was initiated from inside the network.

Q49. Explain the concept of load balancing and name a few common load-balancing algorithms.

Answer:

Load balancing distributes incoming requests across multiple backend servers to avoid overloading any single one and to improve availability. Common algorithms: Round Robin (cycles through servers in order), Least Connections (sends traffic to the server with the fewest active connections), and IP Hash (routes a given client consistently to the same server based on a hash of its IP, useful for session persistence).



Q50. What is a CDN, and how does it reduce latency for end users?

Answer:

A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Q51. Explain the difference between IPv4 and IPv6.

Answer:

IPv4 uses 32-bit addresses (about 4.3 billion possible addresses, written as four decimal octets like 192.168.1.1) and is running out of available addresses. IPv6 uses 128-bit addresses (an astronomically larger space, written in hexadecimal groups like 2001:0db8::1), and also simplifies header processing and has built-in support for features IPv4 needed extensions for, like auto-configuration.

Q52. What is the difference between HTTP and HTTPS, and how does TLS/SSL secure a connection?

Answer:

HTTP sends data in plaintext, so anyone intercepting the traffic can read it. HTTPS wraps HTTP inside a TLS/SSL layer that encrypts the data in transit. Under the hood, a TLS handshake occurs first: the client and server agree on a cipher suite, the server presents a certificate to prove its identity, and both sides derive a shared symmetric session key used to encrypt all subsequent HTTP traffic.

Q53. What is a subnet mask, and how do you calculate the number of usable hosts in a subnet?

Answer:

A subnet mask defines which portion of an IP address is the network part and which is the host part (e.g. 255.255.255.0 means the first 24 bits are network, the last 8 are host). For a /24 subnet, there are $2^8 = 256$ total addresses, minus 2 reserved (network address and broadcast address), giving 254 usable host addresses.

Q54. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q55. What is a VPN, and how does it provide secure remote access over a public network?

Answer:

A VPN (Virtual Private Network) creates an encrypted tunnel between a client and a remote network over a public network like the internet, so traffic appears to originate from the remote network and can't be read by anyone intercepting it in transit. This lets remote employees securely access internal company resources as if they were on the local network.

Q56. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

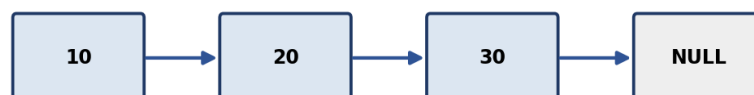
Data Structures & Algorithms

Q57. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in $O(n)$ time and $O(1)$ space.

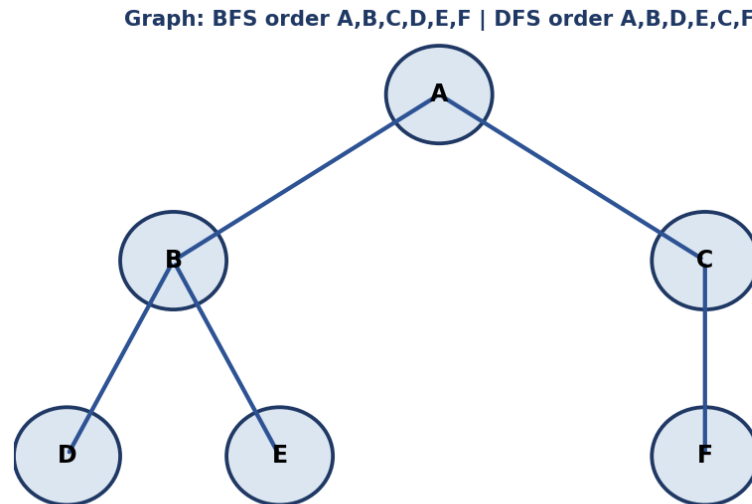
Singly Linked List



Q58. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.

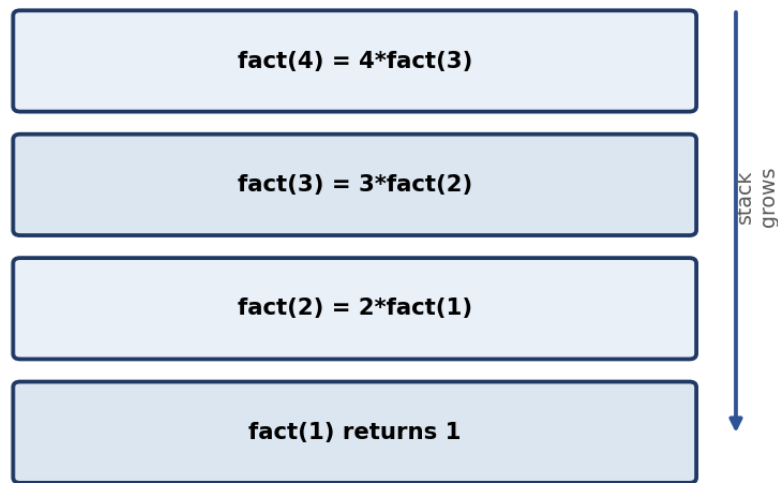


Q59. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.

Call Stack while computing factorial(4)

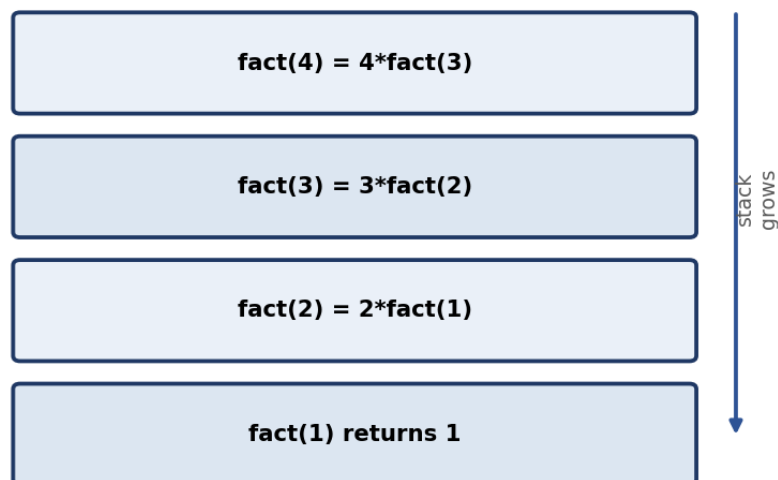


Q60. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.

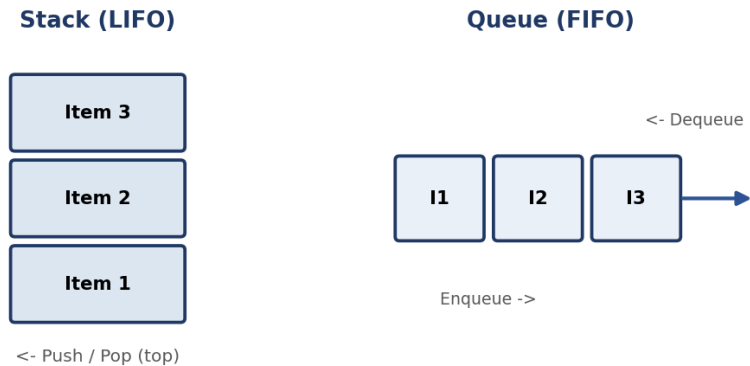
Call Stack while computing factorial(4)



Q61. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q62. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

Answer:

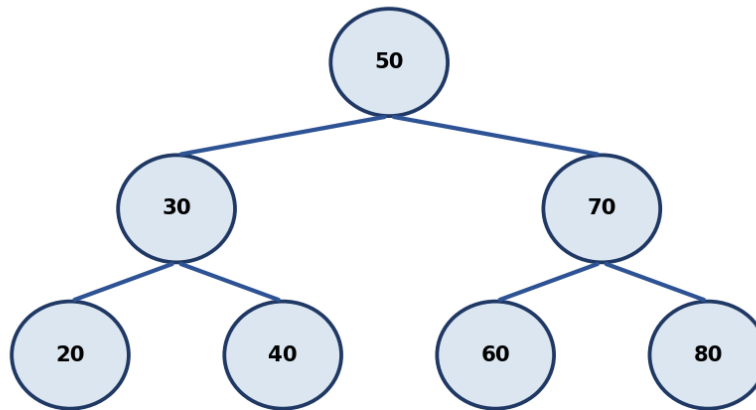
A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q63. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to $\log n$ by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing $O(\log n)$ search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to $O(n)$. Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)

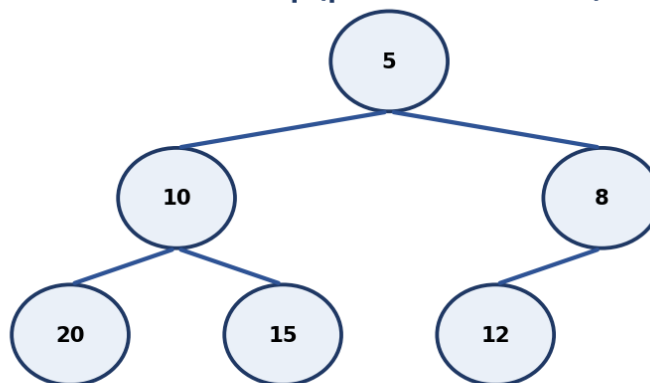


Q64. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take $O(\log n)$ since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.

Min-Heap (parent $\leq$ children)



Q65. Explain the two-pointer technique with an example problem.

Answer:

The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in $O(n)$ instead of the $O(n^2)$ brute force of checking every pair.

Q66. What is the sliding window technique, and in what type of problems is it useful?

Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k , or longest substring without repeating characters), typically turning an $O(n^2)$ brute force into an $O(n)$ solution.

Q67. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in $O(E \log V)$. Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q68. Explain how you would design an LRU cache and the data structures you'd use to achieve $O(1)$ get/put operations.

Answer:

An LRU cache needs $O(1)$ get and put operations. The standard design combines a hashmap (key $\rightarrow$ pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in $O(1)$; when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in $O(1)$, with the hashmap updated accordingly.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q69. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q70. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Coding Problems

Q71. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average $O(n \log n)$, worst case $O(n^2)$ with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed $O(n \log n)$ in all cases, but needs $O(n)$ auxiliary space for merging.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q72. Write a program to remove duplicates from a sorted array in-place.

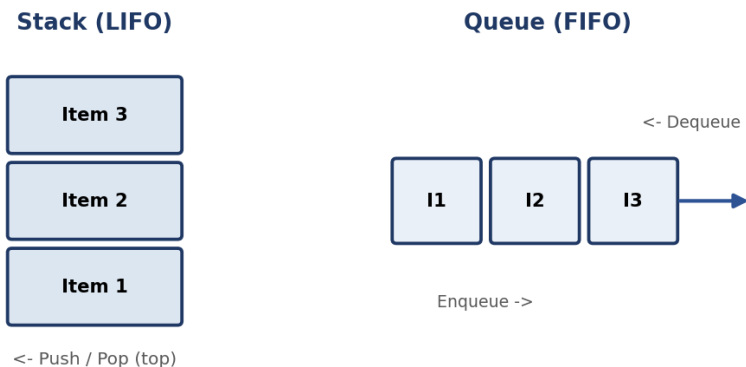
Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in $O(n)$ time and $O(1)$ extra space.

Q73. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized $O(1)$ per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.



Q74. Write a program to find the middle element of a linked list in a single pass.

Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in $O(n)$ time and $O(1)$ space.

Q75. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic ($\text{rear} = (\text{rear} + 1) \% \text{capacity}$) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q76. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

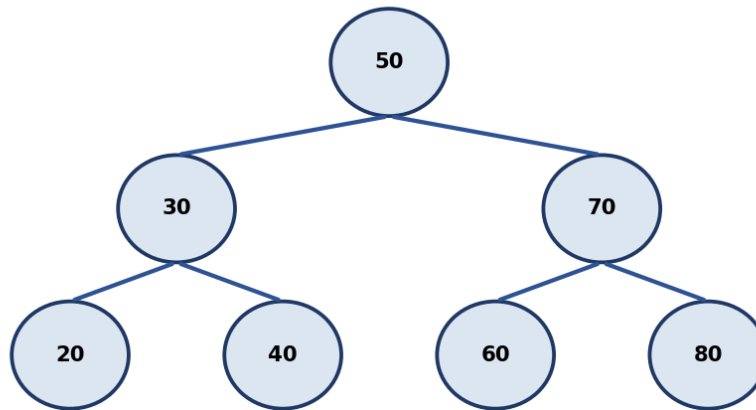
Use a hashmap: iterate through the array once, and for each element x , check if $(\text{target} - x)$ already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in $O(n)$ time and $O(n)$ space, instead of the $O(n^2)$ brute-force nested loop.

Q77. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q78. Write a program to flatten a nested list/array.

Answer:

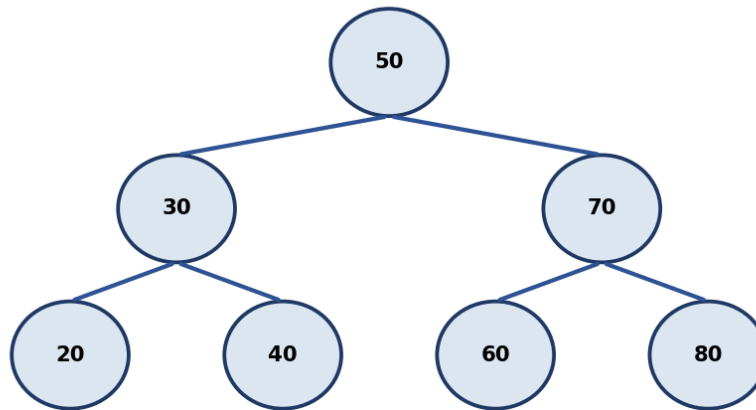
Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity $O(n)$ where n is the total number of elements across all nesting levels.

Q79. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in $O(h)$ time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q80. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table $dp[i][j]$ representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If $A[i-1] == B[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$; otherwise $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. This runs in $O(n*m)$ time and space, where n and m are the string lengths.

Q81. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for $O(1)$ lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in $O(n + m)$ time and $O(\min(n, m))$ space.

Q82. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining $n-k$ elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q83. Write a program to implement binary search on a rotated sorted array.

Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the $O(\log n)$ time complexity of standard binary search despite the rotation.

Q84. Write a program to check whether a binary tree is balanced.

Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving $O(n)$ time instead of the naive $O(n^2)$.

Q85. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Q86. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return -1/None. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Programming Language Fundamentals

Q87. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q88. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q89. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Q90. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q91. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q92. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: `sorted(items, key=lambda x: x.age)`. They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q93. What is the difference between a compiler and an interpreter?

Answer:

A compiler translates the entire source program into machine code (or another target language) ahead of time, producing an executable that runs independently afterward - errors are caught before execution, and execution tends to be faster since translation work is already done. An interpreter reads and executes source code line-by-line (or statement-by-statement) at runtime without producing a standalone executable, which makes development/debugging faster but execution generally slower since translation happens repeatedly during each run.

Q94. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract area() method but a concrete describe() method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing area().

Q95. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. builder.setName('X').setAge(30).build(). To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q96. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via malloc/new), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Web & Cloud Fundamentals

Q97. Explain how a message queue like Kafka or RabbitMQ helps decouple microservices.

Answer:

A message queue like Kafka or RabbitMQ sits between producer services (that generate events/messages) and consumer services (that process them), letting producers publish messages without waiting for consumers to be ready or fast. This decouples services in time and load - producers and consumers can scale, fail, or be deployed independently - and provides buffering so a burst of traffic doesn't overwhelm downstream consumers directly.

Q98. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q99. What is containerization (e.g., Docker), and how is it different from a virtual machine?

Answer:

Containerization packages an application with its dependencies and runtime into a lightweight, portable container (e.g. via Docker) that shares the host OS kernel, making containers fast to start and resource-efficient. A virtual machine, by contrast, virtualizes an entire hardware stack including its own full guest OS, making VMs heavier and slower to start but providing stronger isolation between workloads.

Q100. Explain the difference between authentication and authorization, and how OAuth 2.0 fits in.

Answer:

Authentication verifies who a user is (e.g. checking a username/password or token), while authorization determines what an authenticated user is allowed to do (e.g. which resources/actions they can access). OAuth 2.0 is an authorization framework that lets a user grant a third-party application limited access to their resources on another service (e.g. 'Sign in with Google') without sharing their actual password, typically issuing access tokens that represent specific granted permissions.

Note: 19 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.